

HashCloak

Security Audit Report

Quantus

ML-DSA-87 (Dilithium) Implementation

Auditor: Hernan Dario Vanegas Madrigal – HashCloak Inc.

Prepared for: Yuvi Lightman – Quantus Network

Initial report: March 26, 2026

Updated report: May 28, 2026

Version: 1.2

Audit Commit Hash: `cc5cac8791cf5976a197e7ec68116805f5f4d942`

Post-audit Commit Hash: `fd66916b31b0599aac9f50b504b880765be38b11`

Contents

1 Executive Summary **3**

1.1 Scope of the audit . . . 3

1.2 Code Quality . . . 4

2 Overview of the Audited Schemes **4**

2.1 Notation . . . 4

2.2 ML-DSA-87 . . . 4

2.3 Threshold ML-DSA-87 . . . 5

2.4 Key Generation . . . 5

3 Checklist **6**

3.1 General checks . . . 6

3.2 For threshold ML-DSA-87 . . . 6

4 Findings **8**

Critical HQ1: The DKG protocol is insecure . . . 9

Critical HQ2: Missing verification of commitments for w_i . . . 10

Critical HQ3: Potential wrong generated key . . . 10

Critical HQ4: Corrupted parties can overwrite key contributions . . . 10

Critical HQ5: DKG aggregation accepts unauthorized subset masks . . . 11

Critical HQ6: Missing session identifier in DKG, signing, and resharing . . . 13

Critical HQ7: Nonce reuse in `DilithiumSignProtocol::reset()` . . . 15

Critical HQ8: Signing commitment hash binds only `tr` . . . 16

High HQ9: Missing verification on the signing parties . . . 16

High HQ10: Error-prone API design for secret key . . . 17

High HQ11: DKG seed reuse yields identical Round 1 randomness on retry . . . 18

High HQ12: Silent truncation of `NewShareData` polynomial vectors . . . 19

Medium HQ13: The seed is not zeroized . . . 20

Medium HQ14: Wrong ν parameter . . . 20

Medium HQ15: DoS vector because of silent message drops . . . 21

Medium HQ16: The inner states do not zeroize sensitive information . . . 21

Medium HQ17: Potential overflow in `all_subsets` function . . . 22

Medium HQ18: Silent acceptance of non-participant messages . . . 22

Medium HQ19: Mismatch between `MAX_PARTICIPANTS` and `MAX_PARTIES` . . . 23

Low HQ20: `KeccakState` is `Copy` and is not `Zeroize` . . . 23

Low HQ21: `NoOpSigner` exposed to the public API . . . 24

Informational HQ22: Misleading comments . . . 24

Informational HQ23: Lack of documentation . . . 25

Informational HQ24: Magic numbers . . . 25

Informational HQ25: Presence of debug comments . . . 26

Informational HQ26: “Hedged” test is the same as “non-hedged” test . . . 26

Informational HQ27: Dead code . . . 27

HashCloak

Informational HQ28: Parameter re-definition 27
Informational HQ29: Multiple implementations of NTT 27
Informational HQ30: Unmaintained and unsound advisories from `cargo audit` . . 28

Appendices **29**

A Test Coverage **29**

HashCloak

1 Executive Summary

Quantus Network engaged HashCloak Inc. to review the security of its ML-DSA-87 implementation in two versions: a stand-alone version that implements FIPS-204 [Nat24], and a threshold version that implements the scheme from [Bor+25], along with an implementation of FIPS-202 [Nat15] from scratch. One consultant conducted the assessment from February 16th to March 26th, 2026, using visual inspection, testing, and automated tools, including the MPCSec methodology (`https://mpcsec.org/`) and Claude Opus 4.X as a second opinion[1]. Between March 26th and May 27th, 2026, HashCloak received the fixes and confirmed them for all non-informational issues (see Table 1).

The evaluation, including the post-audit review, surfaced eight critical issues, most of them in the DKG and signing protocols (HQ1, HQ2, HQ3, HQ4, HQ5, HQ6, HQ7, HQ8); four high-severity issues covering missing signing-party checks, an error-prone secret-key API, DKG seed reuse, and silent truncation of resharing shares (HQ9, HQ10, HQ11, HQ12); seven medium-severity issues spanning missing zeroization, wrong parameter values, silent message drops or acceptance, a DKG overflow, and inconsistent participant bounds (HQ13, HQ16, HQ14, HQ15, HQ17, HQ18, HQ19); two low-severity issues (HQ20, HQ21); and several informational issues on code quality, documentation, key management, and dependency advisories (HQ30). Figure 1 summarizes the findings.

As part of this evaluation, we submitted a pull request to the repository, adding more than 300 test cases for the ML-DSA-87 implementation taken from the NIST Cryptographic Algorithm Validation Program (CAVP). It is important to note that this testing does not provide the same level of confidence as the full NIST CAVP, which includes additional test vectors and an official validation process.

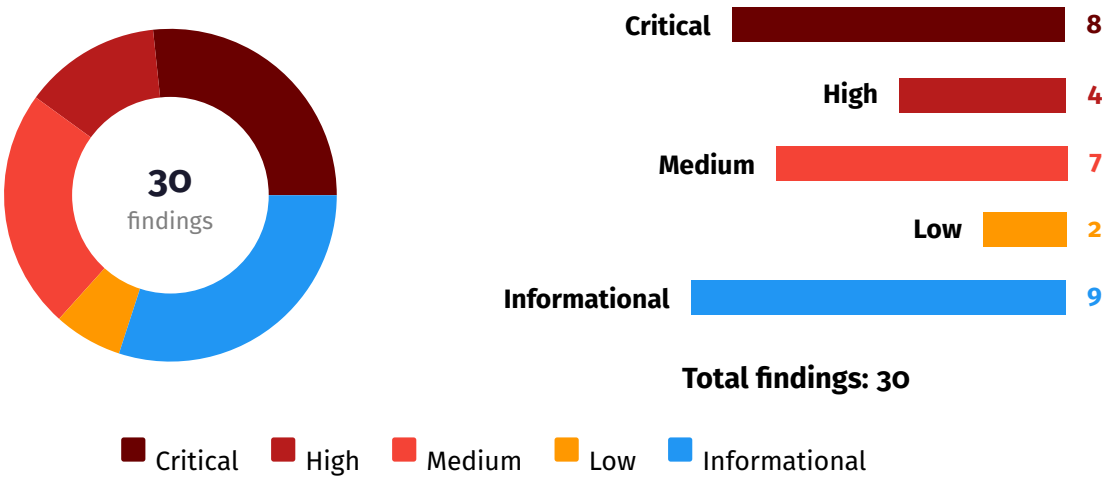


Figure 1: Summary of findings.

1.1 Scope of the audit

HashCloak Inc. audited the code in the repository `https://github.com/Quantus-Network/qp-rusty-crystals` using the commit `cc5cac8791cf5976a197e7ec68116805f5f4d942`. The folders that were specifically audited were: `dilithium`, `threshold`, and `hdwallet`. For this audit, we **excluded** the code in `threshold/src/resharing`.

[1] Claude was **not** used as a black box; the auditor personally verified all results.

1.2 Code Quality

ML-DSA-87 implementation. The main code-quality concern is insufficient documentation for future contributors (see HQ23); in some places it is also misleading (see HQ22). The code follows a C-style and does not exploit Rust features such as generics, member functions, or operator traits, which leads to repetitive code and hurts readability. SHAKE256 and SHAKE128 are also implemented from scratch; while avoiding extra dependencies is the team's call, we recommend a battle-tested implementation to reduce attack surface.

Threshold Dilithium. The threshold implementation has stronger documentation and more idiomatic Rust than the stand-alone one, but still carries some C/Go-style code. Our main concern is that two different NTT implementations coexist — with different representations of the ZETAS array — and are used interchangeably; while we found no correctness issue, this is error-prone for future maintainers.

2 Overview of the Audited Schemes

2.1 Notation

All the notation presented in this report is based on the one presented in [Bor+25]:

- (γ_1, β): Ranges for the signature verification. Here, $\beta = \tau \cdot \eta$, where τ is the number of 1's in the challenge c and η is the secret key range.
- ω: Number of 1's in the hint $\boldsymbol{h}$.
- $\boldsymbol{z}^{(1)}$: Part of the final signature.
- σ: The final signature.
- pk: The public key of the signature scheme.
- $\boldsymbol{w}_i$: Commitment in the ML-DSA-87 signature scheme generated in Round 1 belonging to the party i-th.
- act: Set of parties participating in the signing protocol.
- N: Number of total parties.
- We use the notation $[N]$ to denote the set $\{1, \ldots, N\}$.

2.2 ML-DSA-87

ML-DSA (also known as Dilithium) is a post-quantum secure signature scheme based on the Module Learning With Errors problem. ML-DSA is believed to be secure against adversaries in posession of large-scale quantum computers, and it is also believed to be strongly uno-forgeable, which means that the signature scheme can be used to detect whether some piece of information has unautorized modifications, and non-repudiable, which means that the signature can be used to claim to a third party that it has been generated by the claimed signatory. [Nat24].

ML-DSA is a Schnorr-like signature scheme that uses a Fiat-Shamir heuristic to transform an interactive protocol into a non-interactive one and turn it into a signature scheme. At a high level, the interactive version of ML-DSA consists of the following steps [Nat24]:

- **Commitment:** The prover generates $\boldsymbol{y} \in \mathbb{Z}_q^L$ with small coefficients and commits to its value by sending $\boldsymbol{w}_{\text{Approx}} = \boldsymbol{A}\boldsymbol{y} + \boldsymbol{y}_2$ to the verifier, where $\boldsymbol{y}_2 \in \mathbb{Z}_q^K$ is a vector with small coefficients.
- **Challenge:** The verifier sends a vector $\boldsymbol{c} \in \mathbb{Z}_q^n$ with small coefficients to the prover.
- **Response:** The prover returns $\boldsymbol{z} = \boldsymbol{y} + \boldsymbol{S}_1\boldsymbol{c}$, and the verifier checks that $\boldsymbol{z}$ has small coefficients and that $\boldsymbol{A}\boldsymbol{z} - \boldsymbol{T}\boldsymbol{c} \approx \boldsymbol{w}_{\text{Approx}}$.

To reduce signature size and public key size while providing additional security guarantees, the ML-DSA scheme introduces tweaks to the basic approach, such as module-structured matrices, hint computation, and pseudo-random matrix generation, among others. We refer the reader to the NIST standard [Nat24] for more details.

2.3 Threshold ML-DSA-87

The threshold signing protocol is a three-round scheme that protects against rushing adversaries in the choice of $\boldsymbol{w}$. We present a high level of the rounds from [Bor+25]:

- **First round:** Parties sample randomness $\boldsymbol{r} \leftarrow \chi_r$ and compute the commitment $\boldsymbol{w}_i = \begin{bmatrix}\boldsymbol{A} & \boldsymbol{I}\end{bmatrix} \cdot \boldsymbol{r}_i$. They publish a hash of the commitment $H_{\text{cmt}}(\text{vk}, i, \boldsymbol{w}_i)$.
- **Second round:** Parties reveal the values of $(\boldsymbol{w}_i)_{i \in \text{act}}$.
- **Third round:** Each party computes the response $\boldsymbol{z}_i = c \cdot \boldsymbol{s}_i^{\text{part}} + \boldsymbol{r}_i$, where c is derived from the aggregated commitment $\boldsymbol{w} = \sum_{i \in \text{act}} \boldsymbol{w}_i$ after proper rejection sampling.

After the three rounds, the Combine procedure computes the hint $\boldsymbol{h}$ that allows verifiers to recover the higher bits from $\boldsymbol{w}$ from a value used as an approximation of $\boldsymbol{w}$.

2.4 Key Generation

The audited code contains a protocol intended to generate shares of the secret key and the corresponding public key among the participants. However, after a careful analysis of the implemented protocol, we found that it performs the following steps in four rounds:

- **Round 1:** Each party samples a session ID contribution that will be aggregated with the other parties' contributions with the intention to generate a global session ID to protect against replay attacks.
- **Round 2:** Each party samples $\text{sk}_i = \{s_I \in \mathcal{R}_q \ : \ |I| = N - T + 1 \ \wedge \ I \subseteq [N]\}$ where each s_I has small coefficients, and broadcasts the commitments $c_i = H(\text{sk}_i)$.
- **Round 3:** Each party open the commitments by broadcasting sk_i. Once a party receives a commitment opening, it checks that the received value hash matches the received commitment in Round 2. Here, we have a serious security issue as the participant is revealing its share to the other parties, allowing the other parties to fully reconstruct the secret key. This means that the threshold property of the secret shared key is absent. For more details, see the issue HQ1.
- **Round 4:** Each party aggregates the final contributions by adding the contributions together, then computes the public key on the aggregated values, and finally returns the public key and the aggregated secret key. Here, we also found an important security issue where the addition of contributions with certain distributions does not necessarily follow the same distributions as the operands. This means that the secret key and the public key may have a wrong distribution compared to an ML-DSA-87 standard key pair.

3 Checklist

In this section, we present all the elements that we checked during the audit. Those elements cover from implementation compliance with respect to the specification and practical compliance of security practices, to code and documentation quality.

3.1 General checks

In both threshold implementation and stand-alone implementation, we checked that:

- Given that the ML-DSA implementation can accept inputs for σ or pk of any other length, we checked that it shall return false whenever the lengths of either of these inputs differ from their lengths specified in this standard. Failing to check the length of pk or σ may interfere with the security properties that ML-DSA is designed to have, like strong unforgeability.
- Secret information is zeroized or freed properly in both stand-alone and threshold Dilithium.
- The implementation of ML-DSA shall ensure that any potentially sensitive intermediate data is destroyed as soon as it is no longer needed.
- The implementation of ML-DSA shall not use floating-point arithmetic, as rounding errors in floating-point operations may lead to incorrect results in some cases. Instead, they should use shifting and module operations to guarantee the accuracy of the computations.
- The parameters comply with ML-DSA-87 specification in FIPS-204.
- The rejection sampling for polynomials has the following properties:
  - The result is in the appropriate output range.
  - The rejection rate is negligible: In this implementation, the rejection probability is $\Pr[\text{Rej}] = \frac{2^{13}-2}{2^{23}-1} = 0.000976 = 0.0976\%$.
- The generation of uniform polynomials, vector of polynomials, and matrices matches the specification.
- The documentation is appropriate and accurate.
- The code has good health and reduces the errors for both maintainers and API consumers.

3.2 For threshold ML-DSA-87

For the specific case of Threshold ML-DSA-87, we checked that:

- The signature generated by the protocol is valid using the standard verification of ML-DSA-87.
- The protocol is restarted/aborted if $\|\delta\|_\infty \leq \gamma_2$.
- The protocol is restarted/aborted if $\|\boldsymbol{z}^{(1)}\|_\infty \geq \gamma_1 - \beta$ or if $\boldsymbol{h}$ has more than ω 1's.
- The sampling of shares is done with the correct distribution.
- The state-machine implementation has the correct transitions between states.
- The seeds are generated uniformly at random.
- The implementation of the state machine follows the paper specification.

- The party IDs are in the correct bounds and the parties check that $\mathsf{act} \subseteq [N]$
- The commitments are checked in the second round.

HashCloak

4 Findings

This section details every finding from the audit, grouped by severity. Table 1 summarises all of them at a glance; each ID links to the corresponding finding.

Table 1: Summary of all findings.

ID	Severity	Title	Status
HQ1	Critical	The DKG protocol is insecure	Fixed
HQ2	Critical	Missing verification of commitments for w_i	Fixed
HQ3	Critical	Potential wrong generated key	Fixed
HQ4	Critical	Corrupted parties can overwrite key contributions	Fixed
HQ5	Critical	DKG aggregation accepts unauthorized subset masks	Fixed
HQ6	Critical	Missing session identifier in DKG, signing, and resharing	Fixed
HQ7	Critical	Nonce reuse in `DilithiumSignProtocol::reset()`	Fixed
HQ8	Critical	Signing commitment hash binds only `tr`	Fixed
HQ9	High	Missing verification on the signing parties	Fixed
HQ10	High	Error-prone API design for secret key	Fixed
HQ11	High	DKG seed reuse yields identical Round 1 randomness on retry	Acknowledged
HQ12	High	Silent truncation of `NewShareData` polynomial vectors	Fixed
HQ13	Medium	The seed is not zeroized	Fixed
HQ14	Medium	Wrong ν parameter	Fixed
HQ15	Medium	DoS vector because of silent message drops	Fixed
HQ16	Medium	The inner states do not zeroize sensitive information	Fixed
HQ17	Medium	Potential overflow in `all_subsets` function	Fixed
HQ18	Medium	Silent acceptance of non-participant messages	Fixed
HQ19	Medium	Mismatch between `MAX_PARTICIPANTS` and `MAX_PARTIES`	Fixed
HQ20	Low	`KeccakState` is `Copy` and is not `Zeroize`	Fixed
HQ21	Low	`NoOpSigner` exposed to the public API	Fixed
HQ22	Informational	Misleading comments	Acknowledged
HQ23	Informational	Lack of documentation	Acknowledged
HQ24	Informational	Magic numbers	Acknowledged

Continued on next page

Table 1: Summary of all findings (continued).

ID	Severity	Title	Status
HQ25	Informational	Presence of debug comments	Acknowledged
HQ26	Informational	"Hedged" test is the same as "non-hedged" test	Acknowledged
HQ27	Informational	Dead code	Acknowledged
HQ28	Informational	Parameter re-definition	Acknowledged
HQ29	Informational	Multiple implementations of NTT	Acknowledged
HQ30	Informational	Unmaintained and unsound advisories from `cargo audit`	Acknowledged

HQ1 Critical — The DKG protocol is insecure

File: `threshold/src/keygen/dkg`

Description: In Round 3 of the threshold DKG protocol, each party sends its contribution to everyone as shown in the following excerpt:

```
let msg = DkgRound3Broadcast { party_id: self.my_party_id(), contributions };

self.sent_current_round = true;
Ok(Action::SendMany(self.serialize_message(&DkgMessage::Round3(msg))?))
```

This means that all the parties can reconstruct the final secret key, which is a clear privacy concern. Also, adding elements sampled from a distribution does not guarantee that the sum has the same distribution as the operands. Hence, the strategy of adding contributions may require careful probability analysis.

Recommendations: We recommend using a DKG from the literature. A research result on distributed ML-DSA that includes a DKG protocol is [Bie+25]. The recommended protocol works with any linear secret sharing scheme and inherits the security properties of the secret sharing scheme itself. Hence, if that DKG protocol is instantiated using a linear secret sharing scheme that is secure against malicious adversaries and compliant with the threshold parameter, then the DKG protocol will have the same security guarantees.

It is important to be careful when writing threshold protocols from trusted dealer versions as this task sometimes requires strict and additional security guarantees.

Status: During the post-audit review, this issue was a special case because Quantus Network had to rewrite the DKG protocol to ensure it was correct. Quantus Network implemented the DKG in [Cel+26], Appendix D. To audit this new code, we proceeded in a back-and-forth interaction with Quantus Network, where they brought new code and we checked for new issues in the new code. This was a rolling interaction until we were satisfied with the solutions that Quantus Network proposed. During this interaction, new issues arose, namely, HQ3, HQ4, HQ16, HQ17, HQ18, HQ21. All of these issues were acknowledged and solved. For more information about the specific issues and their solutions, please refer to each one of them in this section.

HQ2 Critical — Missing verification of commitments for w_i

File: `threshold/src/signing.rs`

Description: The current implementation of the protocol does not check that the received w_i from all the other parties matches the commitment cmt_i computed in Round 1. In the paper, this mechanism is designed to avoid rushing adversaries.

This issue is also related to the fact that the function `round2_reveal_with_data` is never called. Hence, not only is this important check missing, but the mechanism that must do the check is never called.

Exploit scenario: The lack of this check allows an adversary to wait for the honest contributions w_i to arrive, and choose its own w_i adaptively, which may bias the challenge in the Fiat-Shamir interaction. Biasing the Fiat-Shamir challenge poses a critical security vulnerability, as the security reduction requires the challenge to be uniformly random, which it is not.

Recommendations: We recommend adding the mentioned checks in the `process_round2` function and calling the function `round2_reveal_with_data` in the third round when all the commitments have been received. In this check, you should compute $H_{\text{cmt}}(\text{vk}, i, w_i)$ and confirm that this matches the received bytes from the network in Round 1.

Status: This issue was acknowledged and solved in the commit `bb02e860979c1e43be9af5761cb721b3885e2dc5`

HQ3 Critical — Potential wrong generated key

File: `threshold/src/keygen/dkg/protocol.rs`

Description: The `complete` function does not ensure that every subset S has a contribution of the key at the end of the protocol. In that case, the variable `all_contributions` may end up with fewer elements than the sets S. This may happen because a corrupted leader may omit some of its partial public key in Round 4 in the code. The consequence is that the protocol may end with the wrong key.

Recommendations: We recommend to assert that `all_partial_pks` has the same elements as `state.config.all_subsets()`.

Status: The issue was acknowledged and solved in the pull request #74.

HQ4 Critical — Corrupted parties can overwrite key contributions

File: `threshold/src/keygen/dkg/protocol.rs`

Description: In the `complete` function, a corrupted party may induce a replacement of an honest contribution to the final key stored in the variable `all_contributions`. This is because the condition $j = \min(S)$ is not being checked in the implementation of Line 6, Procedure `DKGAggregate`, from the DKG protocol in the reference paper.

In particular, the issue can be seen in the following snippet:

```rust
for (&party_id, broadcast) in &state.received_broadcasts {
    // ...
    for (&subset, pk) in &broadcast.partial_public_keys {
        // No check: state.config.get_leader(subset) == Some(party_id)
        let expected = round3.partial_pk_commitments.get(&subset)...;
        let actual = h_commit_pk(subset, pk);
        if actual != *expected { return Err(...); }
        if let Some(&shared_secret) = state.shared_secrets.get(&subset) {
            // ...  only gets in here if the party has K_S
        }
        all_partial_pks.insert(subset, pk.clone());  // BUG: overwrites legit
            entry.
    }
}
```

Recommendations: Once the loop has read all received broadcast messages, assert that `state.config.get_leader(subset) != Some(party_id)`.

Status: The issue was acknowledged and solved in the commit `5f7d654459cbbd8a12af21500d585ccf00685bb8`.

HQ5 Critical DKG aggregation accepts unauthorized subset masks

Files:

- `threshold/src/keygen/dkg/protocol.rs`
- `threshold/src/protocol/partial_pk.rs`

Description: The DKG enforces `is_valid_subset` in only one place — on `Round1Private` (`MithrilRound1Private`) at the following point:

```rust
if !config.is_valid_subset(private.subset_mask) {
    warn!(
        "DKG: Round1Private with invalid subset {:b} (from party {})",
        private.subset_mask, from
    );
    return Ok(()); // Invalid subset, ignore
}
```

However, the Round 3 and 4 maps (`partial_pk_commitments`, `partial_public_keys`) are never filtered against `is_valid_subset`, and `pack_combined_pk` blindly sums **every** value in the map:

```rust
for (&subset, pk) in &broadcast.partial_public_keys {
    // Per Mithril DKGAggregate line 6: only accept PKs from the leader (j =
        min(S))
    let leader = config.get_leader(subset);
    if leader != Some(party_id) {
        // ...
        continue;
```

```
    }
    all_partial_pks.insert(subset, pk.clone());
}
```

```
pub fn pack_combined_pk<'a, I>(rho: &[u8; 32], partial_ts: I) -> PublicKey
where
    I: IntoIterator<Item = &'a [[i32; N as usize]; K]>,
{
    let mut t = polyvec::Polyveck::default();
    for partial in partial_ts {
        for (i, poly_coeffs) in partial.iter().enumerate() {
            for (j, &coeff) in poly_coeffs.iter().enumerate() {
                t.vec[i].coeffs[j] = (t.vec[i].coeffs[j] + coeff) % Q;
            }
        }
    }
```

For non-member verifiers, `verify_partial_pk_commitment` only checks the hash binding when no `shared_secret` is known, so the attacker's tampered `pk_extra` is accepted unconditionally:

```
// If we have the shared secret, verify the PK is correct
if let Some(&shared_secret) = shared_secrets.get(&subset) {
    // ...
}

Ok(())
```

Attack scenario: Consider a 2-of-3 setup. Party 0 is $\min(S)$ for the valid subsets `0b011` and `0b101`, and *also* for the invalid masks `0b001` and `0b111`. Party 0 honestly fills the valid entries and additionally smuggles `0b001` → `pk_extra` with hash-consistent (Round3, Round4). Honest parties (members or not) all pass `verify_party_broadcast`. `pack_combined_pk` sums **4** partial PKs while every honest `PrivateKeyShare.shares` map only encodes **3** subset secrets. The published public key is no longer the key implied by the honest shares — signing produces nothing that verifies under it, and the key is effectively dead. With a more careful choice of t_{extra}, party 0 also introduces an additive offset it knows entirely, biasing the global key toward a value it controls.

Recommendation: In `collect_and_verify_all_partial_pks`, reject any `subset` where `config.is_valid_subset(subset) == false`, both when reading the local `my_partial_pks` and when reading Round 4 broadcasts. The same check should be applied to `partial_pk_commitments` in Round 3.

Status: This issue has been acknowledged and solved in commit `d2f7e1a20874895544529aa41b14bf60cede6c8e`

HashCloak

HQ6 Critical Missing session identifier in DKG, signing, and resharing

Files:

- threshold/src/keygen/dkg/types.rs
- threshold/src/protocol/signing.rs
- threshold/src/signing_protocol.rs
- threshold/src/resharing/protocol.rs

Description: The DKG commitment hash binds only the domain, the party ID, and the data:

```
pub fn h_commit(party_id: ParticipantId, data: &[u8]) -> [u8;
    COMMITMENT_HASH_SIZE] {
    let mut state = fips202::KeccakState::default();
    fips202::shake256_absorb(&mut state, DOMAIN_COMMIT, DOMAIN_COMMIT.len());
    fips202::shake256_absorb(&mut state, &party_id.to_le_bytes(), 4);
    fips202::shake256_absorb(&mut state, data, data.len());
    // no ssid, no (t,n), no participant list
```

The function `h_commit_pk` is even weaker — it does not bind `party_id`:

```
pub fn h_commit_pk(
    subset_mask: SubsetMask,
    partial_pk: &PartialPublicKey,
) -> [u8; COMMITMENT_HASH_SIZE] {
    // DOMAIN_PK_COMMIT || subset_mask || pk.t   <- no party_id, no ssid
```

The DKG transcript hash binds only the per-round broadcasts:

```
pub fn compute_transcript_hash(
    round1_broadcasts: &BTreeMap<ParticipantId, MithrilRound1Broadcast>,
    round2_broadcasts: &BTreeMap<ParticipantId, MithrilRound2Broadcast>,
    round3_broadcasts: &BTreeMap<ParticipantId, MithrilRound3Broadcast>,
) -> [u8; 32] {
    // rounds 1/2/3 only -- no (t,n), no participants, no ssid
```

The signing commitment hash binds only (`tr`, `party_id`, `commitment_data`):

```
pub(crate) fn compute_commitment_hash(
    tr: &[u8; 64],
    party_id: ParticipantId,
    commitment_data: &[u8],
) -> [u8; 32] {
    // tr || party_id || commitment_data  <- no ssid, no participant set
```

A design comment **acknowledges and refuses to fix this**:

```
//! 1. The Mithril paper (Section 2.2) defines threshold signatures *without*
    session identifiers,
```

```
//!    relying on each honest party's local state to enforce per-attempt
    freshness. In-protocol
//!    retries on a single instance violate that assumption because the
    receiver's state machine has
//!    no way to bind incoming rounds to "the current attempt" without an
    explicit session id, which
//!    the wire format does not carry.
```

The resharing session seed (Round 1 and 2) is derived only from per-party entropy:

```
fips202::shake256_absorb(&mut state, SESSION_SEED_DOMAIN, SESSION_SEED_DOMAIN.
    len());
for (&party_id, entropy) in &sorted_parties {
    fips202::shake256_absorb(&mut state, &party_id.to_le_bytes(), 4);
    fips202::shake256_absorb(&mut state, *entropy, ENTROPY_SIZE);
}
// no public_key, no (old_t,n), no (new_t,n), no ssid
```

Attack scenario: This attack class mirrors CVE-2022-47930 in `tss-lib`:

1. **DKG transcript replay.** Two DKG runs with the same participant set, the same seeds, and the same (t, n) produce identical transcripts. Transcript signatures from run A are structurally valid in run B.
2. **Signing cross-session pid spoof.** The only thing identifying a commitment is $(\texttt{tr}, \texttt{party_id}, \texttt{commitment_data})$. The value `tr` is the public-key hash and is **fixed** for the life of the key. Combined with any seed-reuse bug, the same $(\texttt{commitment_hash}, \texttt{commitment_data})$ pair from session A is valid in session B without modification. A rushing adversary that observed an honest party H in session A holds $(c_H, \boldsymbol{w}_H)$ and can replay it if H ever regenerates with the same seed.
3. **Concurrent signing sessions.** Nothing in the wire format prevents two simultaneous signing sessions for the same key from being interleaved — a Round 2 reveal for session X verifies in session Y because no SSID is in the hash. This is exactly the "rushing across sessions" attack class.

Recommendation: Define a canonical session identifier

$$\text{ssid} = H(\text{ssid} \,\|\, \text{PublicKey} \,\|\, \text{ParticipantList} \,\|\, (t, n) \,\|\, \text{AttemptNonce}),$$

and prepend it to every commitment hash, Fiat-Shamir challenge, transcript, and signing message. The transport-layer `ChannelId` freshness signal must be promoted into a cryptographic one.

Status: This issue has been acknowledged and solved in `fd66916b31b0599aac9f50b504b880765be38b11`.

HashCloak

HQ7 Critical Nonce reuse in `DilithiumSignProtocol::reset()`

File: `threshold/src/signing_protocol.rs`

Description: The function `reset()` clears all per-round state but keeps the existing `round1_seed`:

```rust
pub fn reset(&mut self) {
    self.state = SignProtocolState::Round1Generate;
    self.r1_broadcasts.clear();
    self.r2_broadcasts.clear();
    self.r3_broadcasts.clear();
    self.my_r1 = None;
    self.my_r2 = None;
    self.my_r3 = None;
    self.message_buffer.clear();
    self.received_signature = None;
    self.signer.reset();
}
```

The module documentation warns about this:

```rust
/// Prefer constructing a fresh `DilithiumSignProtocol` over calling `reset()`.
/// `reset()` keeps the same `round1_seed`, so a subsequent attempt on the same
/// instance will reuse identical Round 1 randomness -- which is **only safe**
/// when the previous attempt never sent a Round 1 message on the wire.
```

However, no code enforces this constraint. Any caller that invokes `reset()` after at least one `poke()` causes the threshold signer to regenerate the identical hyperball sample y_i for the new attempt.

Attack scenario: In ML-DSA, the response is linear in the secret share: $z_i = c \cdot s_i + y_i$. Two attempts on the same message and committee using the same y_i are equivalent and leak nothing. However, the moment any honest party's perceived message, context, or aggregated w differs across two attempts (so that the challenge c differs), the attacker recovers s_i from

$$z_i^{(1)} - z_i^{(2)} = (c_1 - c_2) \cdot s_i .$$

Differing-message replay is exactly what a malicious leader can induce by delivering different `message` parameters to different parties across two attempts on the same instance. Even more directly, the lack of an SSID (see HQ6) means a malicious peer can deliver different `r2_broadcasts` (i.e., different aggregated w) across two attempts on the same `round1_seed`, leaking s_i from one party's two responses. This is the canonical presignature-reuse failure mode from [**EPRINT:GenGol18**; **IA:BMSS10**], repurposed for ML-DSA-87 shares.

Recommendation: The function `reset()` must either take a new seed parameter (`reset(&mut self, new_seed: [u8; 32])`) or panic / return an error when called after the first `poke()`. Mark the current method as `unsafe` / test-only, or, preferably, delete it.

Status: The function `reset()` was removed entirely. This issue was acknowledged and solved in commit `94650a0358dd5b5849a5d958aa8bf3f3e59018b1`.

HashCloak

HQ8 Critical — Signing commitment hash binds only `tr`

File: `threshold/src/protocol/signing.rs`

Description: The signing commitment hash binds only (`tr`, `party_id`, `commitment_data`):

```
pub(crate) fn compute_commitment_hash(
    tr: &[u8; 64],
    party_id: ParticipantId,
    commitment_data: &[u8],
) -> [u8; 32] {
```

The value `tr` is the public-key hash and is identical across all signing sessions for the same key. Two sessions of the same key are therefore indistinguishable to `compute_commitment_hash`. The only thing preventing an adversary from replaying an honest party's prior $(c_H, \boldsymbol{w}_H)$ is that the honest party must regenerate fresh $(\boldsymbol{y}, \boldsymbol{e})$ per session, which is enforced **only** by seed freshness in the caller (see HQ7 and HQ11).

Attack scenario: This is the same attack class as CVE-2022-47930. If seed freshness ever lapses — via HQ7, via an external restart that reuses the seed, or via any reuse of the Round 1 seed — the commitment hash from session A becomes a valid commitment in session B without modification, because `tr` is the only session-dependent value bound into the hash and it is constant for the life of the key. A rushing adversary that observed an honest party's $(c_H, \boldsymbol{w}_H)$ in any prior session of the same key can replay it in a new session, biasing the Fiat-Shamir challenge.

Recommendation: Bind a canonical session identifier (see HQ6) into `compute_commitment_hash`, and additionally bind the participant set. With ssid included, sessions are cryptographically distinguishable independently of the caller's seed-management discipline.

Status: The issue has been acknowledged and solved in `fd66916b31b0599aac9f50b504b880765be38b11`.

HQ9 High — Missing verification on the signing parties

File: `threshold/src/signer.rs`

Description: The specification in the paper (Figure 5) suggests to check the following conditions:

- act $\subseteq [N]$,
- $i \in$ act.

However, the code in Round 2, implemented in the function `round2_reveal`, does not check for these conditions.

Attack scenario: The possible effects of not checking these conditions are the following:

- Given that the party ID is used in Round 3, if $i > N$ then the bit would be outside of the valid range, causing an incorrect recovery.
- If $i > 16$, this will overflow the `u16` shift causing a runtime panic and potential DoS.

- There is no check that the current party is included in the active set of signing parties, which may cause a silent protocol failure in Round 3.

Recommendation: We suggest checking the IDs in the messages received from Round 1 are in the correct bounds, that is, the IDs are less than 16, and the IDs are less than N, as follows

```
if self.party_id() >= 16
    || self.party_id() >= self.config().total_parties()
    || self.active_parties().contains(self.party_id())
{
    return Err(ThresholdError::InvalidPartyId {
        party_id: self.party_id(),
        max_id: self.config().total_parties().min(16),
    });
}
```

Status: This issue was acknowledged and solved in the commit `bb02e860979c1e43be9af5761cb721b3885e2dc5`

HQ10 High — Error-prone API design for secret key

Files:

- `dilithium/src/ml_dsa_87.rs`

Description: The `bytes` field in the struct `SecretKey` is `pub`. Given that the struct derives `ZeroizeOnDrop`, a caller function may execute the line

```
let sk_tmp = sk.bytes
```

At this point, the benefits of `ZeroizeOnDrop` do not apply to `sk_tmp`, and the caller must be sure that it is zeroizing `sk_tmp` at the end of its use. Although this does not affect security immediately, it may increase the risks when using the API in a wider context.

Another potential issue is that the function `SecretKey::to_bytes` returns an array, which is `Clone`. Hence, zeroization is the caller's responsibility, and failure to do so properly may be a potential issue.

Recommendation: We suggest enclosing the returned bytes in a wrapper similar to `SensitiveBytes32`, so that the data structure has the same patterns and zeroizing mechanisms to enforce security by design. A less error-prone solution would be to make the `bytes` field private, remove the `Clone` derive, and change the `to_bytes` to return a reference as follows:

```
/// Private key.
#[derive(ZeroizeOnDrop)]
pub struct SecretKey {
    bytes: [u8; SECRETKEYBYTES],
}

impl SecretKey {
```

```
    /// Returns a slice of the underlying bytes.
    pub fn as_bytes(&self) -> &[u8; SECRETKEYBYTES] {
        &self.bytes
    }

    // ---<snip>---
}
```

This does not prevent the user from having multiple copies of the secret key bytes completely for an API user. However, this reduces the set of ways the API consumer can inadvertently obtain the bytes, making it more difficult. In this way, the API consumer is more mindful when copying secret key information.

Status: This issue was acknowledged and solved in the commit `bb02e860979c1e43be9af5761cb721b3885e2dc5`

HQ11 High DKG seed reuse yields identical Round 1 randomness on retry

File: `threshold/src/keygen/dkg/protocol.rs`

Description: The Round 1 randomness for each party is derived deterministically from a 32-byte `seed` supplied by the caller at instantiation:

```
fn derive_round1_randomness(
    seed: &[u8; 32],
    party_id: ParticipantId,
    leader_subsets: &[SubsetMask],
) -> ([u8; RANDOMNESS_SIZE], BTreeMap<SubsetMask, [u8; SHARED_SECRET_SIZE]>) {
```

Reusing the same `seed` after a failed run produces the same (r_i, K_S, R, ρ). The protocol does not record seeds it has used before, nor does it bind the seed to a unique session identifier at instantiation.

Attack scenario: Combined with the absence of an SSID (see HQ6), an attacker who recorded any partial Round 2 reveal from a failed run can replay it verbatim in the retry. An honest party that restarts from a seed already observed by the adversary surrenders the same randomness twice, allowing the adversary to replay partial reveals across the two runs without modification. This is the canonical backup-and-restore failure mode for threshold protocols, where state captured before the abort remains usable against the same randomness after the restart.

Recommendation: Bind the seed at instantiation to a session-unique nonce (typically the canonical SSID), and refuse to construct a DKG with a `seed` that has been used before. A persistent record of used seeds at the caller boundary is the safest enforcement point; in-protocol, at minimum, mix the SSID into `derive_round1_randomness` so that two runs with the same `seed` but different SSIDs still produce different r_i.

Status: This issue was acknowledged but is out of scope given that Quantus Network claims that the execution will be within NEAR protocol which handles the seed generation for each session.

HQ12 High Silent truncation of `NewShareData` polynomial vectors

Files:

- `threshold/src/resharing/types.rs`
- `threshold/src/resharing/protocol.rs`

Description: The structure `NewShareData` carries the per-share polynomials as length-unconstrained `Vec`s:

```
pub struct NewShareData {
    pub s1: Vec<[i32; N as usize]>,
    pub s2: Vec<[i32; N as usize]>,
}
```

The aggregation function `add_share_into` pairs the two structures through `zip`:

```
fn add_share_into(acc: &mut NewShareData, r: &NewShareData) {
    for (a, b) in acc.s1.iter_mut().zip(r.s1.iter()) {
        for (ac, bc) in a.iter_mut().zip(b.iter()) {
            *ac = ac.wrapping_add(*bc);
```

The standard-library `zip` silently truncates whenever `r.s1.len() != L` or `r.s2.len() != K`. Later code zero-pads missing polynomials, so a short `r` silently contributes zero to the trailing polynomials.

Attack scenario: A malicious dealer in the resharing protocol can ship a `NewShareData` whose `s1` or `s2` vector is shorter than L or K. The receiver passes the deserialization (no length check is enforced), accepts the truncated vector, and aggregates zeros into the trailing polynomials. The resulting share is silently corrupted while every hash binding still verifies for the single-member RSS targets the malicious dealer chose. The corrupted share will not produce valid signatures, but the corruption is invisible until the first signing attempt, at which point the protocol must abort or reshare again.

Recommendation: In the deserializer for `NewShareData`, enforce `s1.len() == L && s2.len() == K`. The same exact-length enforcement should be applied at deserialization to every fixed-shape vector received over the network.

Status: The issue was acknowledged and solved in commit `0ebd321c074ad2b455c1fa042445 32d62f8c14dd`.

HQ13 Medium — The seed is not zeroized

File: `threshold/src/signing.rs`

Description: The seed variable `seed` that is used to compute the commitment is not zeroized. Although the variable is an array stored on the stack and goes out of scope at the end of the `round1_commit` function, it is recommended to avoid leaking sensitive information.

Attack scenario: A potential attack could be that an adversary reads residual stack memory via a memory disclosure vulnerability. In that case, the adversary will learn the commitment r_i and extract the secret share from the information in the third round as $s_i^{\text{part}} = (z_i - r_i) \cdot c^{-1}$. In a memory disclosure vulnerability, an adversary can access residual memory after it is no longer needed by the process executing the protocol. For example, if the protocol is executing and the secret data is stored in swap memory, it can remain there even after the process finishes. In that case, the adversary can access that part of the memory and retrieve the seed and, consequently, retrieve the share of the party owning that process.

Recommendation: We suggest either manually zeroize the array, but a stronger and idiomatic solution would be to create a type `Seed([u8; 32])` that derives the trait `ZeroizeOnDrop` or use the `SensitiveBytes32` that is already implemented in the library. The simplest way to remedy this issue is to add the line `seed.zeroize()` before returning the resulting value of the function at the end of `round1_commit`.

Status: This issue was acknowledged and solved in the commit `bb02e860979c1e43be9af5761cb721b3885e2dc5`

HQ14 Medium — Wrong ν parameter

File: `threshold/src/signing.rs`

Description: The ν parameter returned by the function `get_threshold_params` is $\nu = 3$, but the paper establishes that $\nu = 7$ for ML-DSA-87. Given that the parameters r and r' are set for the specific value of ν, this may affect the success probability from its aimed value of $\approx 1/2$ for one protocol execution.

Recommendation: Change the function `get_threshold_params` for the following version

```
fn get_threshold_params(config: &ThresholdConfig) -> (f64, f64, f64) {
    match (config.threshold(), config.total_parties()) {
        (2, 2) => (503119.0, 503192.0, 7.0),
        (2, 3) => (631601.0, 631703.0, 7.0),
        (3, 3) => (483107.0, 483180.0, 7.0),
        (2, 4) => (632903.0, 633006.0, 7.0),
        (3, 4) => (551752.0, 551854.0, 7.0),
        (4, 4) => (487958.0, 488031.0, 7.0),
        (2, 5) => (607694.0, 607820.0, 7.0),
        (3, 5) => (577400.0, 577546.0, 7.0),
        (4, 5) => (518384.0, 518510.0, 7.0),
        (5, 5) => (468214.0, 468287.0, 7.0),
        (2, 6) => (665106.0, 665232.0, 7.0),
        (3, 6) => (577541.0, 577704.0, 7.0),
        (4, 6) => (517689.0, 517853.0, 7.0),
```

```
        (5, 6) => (479692.0, 479819.0, 7.0),
        (6, 6) => (424124.0, 424197.0, 7.0),
        _ => (503119.0, 503192.0, 7.0),
    }
}
```

Status: This issue was acknowledged and solved in the commit `bb02e860979c1e43be9af5761cb721b3885e2dc5`

HQ15 Medium — DoS vector because of silent message drops

File: `threshold/src/signing_protocol.rs`

Description: When a party receives and processes an incoming message using the `message` function, it does not alert the node to a malformed or wrong message but silently drops the message.

Attack scenario: An adversary can send a malformed message continuously to a victim node so that the latter freezes because the message is being dropped, but there is no liveness parameter, notification, or maximum number of messages received that allows the party to either abort or continue the protocol. This is, in the end, a DoS vulnerability.

Recommendation: A robust and network-independent solution may be changing the `message` function to return a `Result<T, E>` so that when the message is malformed it return a proper error and this could be handled outside using either a parameter to allow certain number of malformed messages before panicking or a time parameter to abort the execution after certain amount of inactivity. A correct version of the deserialization is

```rust
let msg = match self.deserialize_message(&data) {
    Ok(m) => m,
    Err(_) => Err(SignProtocolError::InvalidMessage(data)),
};
```

Then this error is handled by the network layer according to the liveness parameter.

Status: This issue was acknowledged and solved in the commit `bb02e860979c1e43be9af5761cb721b3885e2dc5`

HQ16 Medium — The inner states do not zeroize sensitive information

Files:

- `threshold/src/keygen/dkg/state.rs`
- `threshold/src/keygen/dkg/protocol.rs`

Description: The states in the DKG protocol state machine are not being zeroized. In particular, the values of K_S sent by the leaders of each group S are private information and should be zeroized after use.

Recommendations: We recommend to either zeroize the map `MithrilRound1State::my_shared_secrets` manually or implement the `ZeroizeOnDrop` trait for `MithrilRound1State`.

Status: This issue was acknowledged and solved in the following commits:

- `fd211ace9074839d2c66ffef77b06d8b2855eab5`,
- `9ba78abcc8ca6495a62a73d568cac9173f427631`,
- `283b0a84f4fe23a7147fe8b269a0b168e3514da5`, and
- `4ab00866bc8981b97e9b164dacc5e30a62935d89`

HQ17 Medium — Potential overflow in `all_subsets` function

File: `threshold/src/keygen/dkg/types.rs`

Description: If $n > 16$, the `all_subsets` function will have an overflow and panic because `mask` variable is a `u16`.

Recommendations: Either, ensure that $n \leq 16$ returning an error in case that $n > 16$, or use a type that allows you to hold more bits.

Status: This issue was acknowledged and solved in the following commits

- `116d3f33f7ac5f8542c030dbe06134b499366158`, and
- `ea92d90c0799c0375903087f2c0026df59206454`.

HQ18 Medium — Silent acceptance of non-participant messages

File: `threshold/src/keygen/dkg/protocol.rs`

Description: The `message` function does not check that the party sending the message is in fact a party from the set of participating parties. Although this error is captured in Round 2, it affects the protocol's liveness by returning an error that is completely different at a late stage of execution.

Recommendations: We recommend to add the following assertion in the `message` function.

```
if !state.config.all_participants.contains(&from) || from == state.config.
    my_party_id {
    return Err(...);
}
```

Status: This issue was acknowledged and solved in the commit `c758bb112b01507be9f773d79689fa8a2cc92a89`.

HashCloak

HQ19 Medium Mismatch between MAX_PARTICIPANTS and MAX_PARTIES

Files:

- threshold/src/participants.rs
- threshold/src/error.rs

Description: Two different upper bounds on the number of parties coexist in the crate. The participant-list code uses 16:

```
pub const MAX_PARTICIPANTS: usize = 16;
```

while the threshold-configuration code uses 6:

```
pub const MAX_PARTIES: u32 = 6;
```

The function ParticipantList::new accepts up to 16 IDs and ThresholdConfig::new enforces $n \leq 6$. However, ResharingConfig and several downstream call sites build ParticipantLists directly and accept the wider range. Combined with the "Maximum 15 parties supported" constraint inside compute_sharing_patterns, a caller can construct internally inconsistent configurations that pass one validator and fail (or, worse, partly succeed) at another.

Attack scenario: The mismatch lets a caller — malicious or simply mistaken — assemble a protocol instance whose participant set has a different cardinality than the threshold configuration believes. Downstream code that reads from the wider bound (16 or 15) may iterate over entries that the threshold path never accounted for, while code that reads from the narrower bound (6) may silently drop participants that the participant list considered active. Each component then believes it is operating on a different protocol instance than the others, producing exactly the "got a different protocol instance than I thought" failure class.

Recommendation: Thread MAX_PARTIES through every entry point so that any participant-count parameter is rejected uniformly. The constraints inside ParticipantList::new, ThresholdConfig::new, ResharingConfig, and compute_sharing_patterns must all agree on a single upper bound, ideally enforced by a shared constructor that the looser entry points are required to call.

Status: This issue has been acknowledged and solved in commit 8f2a9726203dd74ca9b8b24051accf4c12fdbbe8

HQ20 Low KeccakState is Copy and is not Zeroize

Files:

- dilithium/src/fips202.rs

Description: The KeccakState struct does not have a zeroization on drop, and it derives Copy. This allows two types of misbehaviors during its use:

1. Given that `KeccakState` is `Copy` state can be copied in without any problem as

```
let state = keccak_state;
```

   Leaving the private Keccak state alive in intermediate variables.

2. Given that `KeccakState` is not zeroized, and that the state may hold information about the secret key, this sensitive information may stay in the memory.

Status: This issue was acknowledged and solved in the commit `bb02e860979c1e43be9af5761cb721b3885e2dc5`

HQ21 Low — `NoOpSigner` exposed to the public API

File: `threshold/src/keygen/dkg/protocol.rs`

Description: The struct `NoOpSigner` is exposed as `pub` knowing that this struct is a signer that accepts any signature and is used for testing purposes. Leaving this structure publicly accessible to end users may have catastrophic consequences if used.

Recommendation: Restrict the struct `NoOpSigner` to be used only for testing purposes by adding a Cargo feature called `test-utils`. Therefore, any end user who wants to use this struct should enable the feature at their own risk.

Status: This issue was solved and acknowledged in the commit `130c65ab1e3b611cab9bc64089802b0d5a375d6e`.

HQ22 Informational — Misleading comments

Files:

- `dilithium/src/poly.rs`
- `dilithium/src/rounding.rs`
- `threshold/src/primitives.rs`

Description: The following functions have wrong or misleading comments:

- `rej_uniform`: The name of the parameters is incorrect.
- `decompose_go`: The comment in this method does not make a difference between α and a in the text. Fixing this could increase readability.
- `power2round` in `dilithium/src/poly.rs`: The comment does not specify which is the input polynomial and uses the incorrect variable names in the documentation comments.
- `decompose`: The documentation in the function does not specify what is `ALPHA`, and this variable does not appear inside the function implementation. Given that `ALPHA` is not specified, this causes the function to contain magic numbers that make its readability difficult (see issue HQ24).

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ23 Informational — Lack of documentation

Description: The following files in the ML-DSA-87 implementation contain at least one element without documentation:

- `dilithium/src/errors.rs`
- `dilithium/src/fips202.rs`
- `dilithium/src/ml_dsa_87.rs`
- `dilithium/src/params.rs` – All 25 public constants are undocumented.
- `dilithium/src/polyvec.rs`
- `dilithium/src/sign.rs`
- `dilithium/src/reduce.rs`
- `dilithium/src/poly.rs`
- `dilithium/src/lib.rs`
- `dilithium/src/packing.rs`

Also, the following files in the Threshold ML-DSA-87 implementation contain at least one element without documentation:

- `threshold/src/circl_ntt.rs`
- `threshold/src/serde_helpers.rs`
- `threshold/src/protocol/secret_sharing.rs`

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ24 Informational — Magic numbers

Files:

- `dilithium/src/poly.rs`
- `dilithium/src/decompose.rs`
- `threshold/src/primitives.rs`

Description: There are some constants without a clear description:

- In `dilithium/src/poly.rs`, the variable `UNIFORM_NBLOCKS` has an operand with value 767, which does not have a clear description.
- In `dilithium/src/rounding.rs`, the `decompose` function has some numbers that are not named or documented: 1025, 22, and 15.

- In `threshold/src/rounding.rs` it would be better to express `18446744073709551616` as `1 « 64`.

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ25 Informational — Presence of debug comments

Files:

- `dilithium/src/fips202.rs`

Description: There are lines that are commented out that, if they were uncommented by accident in a production deployment, could reveal information about the secret key.

```
// println!("i = {}", i);
// println!("out[idx] przed przypisaniem: {}", out[idx]);
out[idx] = (s[i / 8] >> 8 * (i % 8)) as u8;
// println!("out[idx] po przypisaniu: {}", out[idx]);
```

The state could hold information about the secret key that is then moved to the `out` array.

Suggestion: Remove the commented code to avoid human mistakes.

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ26 Informational — "Hedged" test is the same as "non-hedged" test

Files:

- `dilithium/src/ml_dsa_87.rs`

Description: The tests `self_verify` and `self_verify_hedged` are the same. The correct test for `self_verify` would be:

```
#[test]
fn self_verify() {
    let msg = get_random_msg();
    let entropy = get_random_bytes();
    let keys = Keypair::generate(entropy);
    let sig = keys.sign(&msg, None, None).unwrap();
    assert!(keys.verify(&msg, &sig, None));
}
```

This is code duplication and a missing coverage of the correct code.

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ27 Informational — Dead code

File: `dilithium/src/poly`

Description: The probability that the while loop in function `uniform` is executed has a probability of 1×10^{-39}. Hence, the following code is statistically dead code for the parameter set of ML-DSA-87:

```
while ctr < N {
    let off = buflen % 3;
    for i in 0..off {
        buf[i] = buf[buflen - off + i];
    }
    buflen = fips202::SHAKE128_RATE + off;
    fips202::shake128_squeezeblocks(&mut buf[off..], 1, &mut state);
    ctr += rej_uniform(&mut a.coeffs[ctr..], N - ctr, &buf, buflen);
}
```

This code is responsible for generating additional randomness when the previously generated randomness is not sufficient. But statistically speaking, this situation is very unlikely to happen.

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ28 Informational — Parameter re-definition

File: `threshold/src/protocol/primitives.rs`

Description: The parameters from threshold ML-DSA-87 are defined again in the crate. If the goal is to use the parameters of the stand-alone ML-DSA-87 implementation, we recommend using those parameters. In that way, the protocol is less prone to parameter-definition errors.

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ29 Informational — Multiple implementations of NTT

Files:

- `threshold/src/circl_ntt.rs`
- `dilithium/src/ntt.rs`

Description: The repository contains two implementations of the NTT algorithm using different representations. Although the array of `dilithium::ntt::ZETAS` differs from `threshold::circl_ntt::ZETAS` in the index `0` when computed modulo `Q`, it does not affect the correctness of the algorithms as `ZETAS[0]` is never indexed. In this case, the array `ZETAS` stores the values $\zeta_i = \zeta^{2i+1}$, where ζ is a 512-th root of unity.

Given that both implementations use different representations of the underlying NTT domain, interoperability conflicts between the two versions of NTT may increase the likelihood of incor-

rect use. To reduce the possibility of this error, we recommend unifying both implementations to use a single set of NTT algorithms.

Status: This issue was acknowledged but, to our knowledge, has not been solved.

HQ30 Informational Unmaintained and unsound advisories from `cargo audit`

Files:

- `Cargo.lock`

Description: Running `cargo audit` against the workspace surfaces six advisories. None correspond to a direct dependency of the audited crates — all are pulled in transitively, the majority through the development-only `dudect-bencher` crate (which itself depends on the legacy `clap 2.34`) or through the Plonky3 family (`p3-*`) used by `qp-poseidon-core`. The advisories are:

- **RUSTSEC-2021-0139** — `ansi_term 0.12.1` is unmaintained. Pulled in via `clap 2.34` → `dudect-bencher`.
- **RUSTSEC-2024-0375** — `atty 0.2.14` is unmaintained. Pulled in via `clap 2.34` → `dudect-bencher`.
- **RUSTSEC-2021-0145** — `atty 0.2.14` contains a potential unaligned read. Same dependency path; this is a development-only path used for constant-time benchmarking of the ML-DSA-87 implementation.
- **RUSTSEC-2024-0436** — `paste 1.0.15` is no longer maintained. This is a compile-time procedural macro pulled in through `p3-field`, `p3-goldilocks`, and related Plonky3 crates; it has no runtime surface.
- **RUSTSEC-2026-0097** — `rand 0.8.5` and `rand 0.9.2` are reported as unsound when used with a custom logger via `rand::rng()`. Both versions are direct dependencies of crates in the workspace.

Attack scenario: The unmaintained and unaligned-read advisories against `ansi_term`, `atty`, and `paste` are not exploitable from the production binary: `ansi_term` and `atty` appear only on a development-only dependency path (`dudect-bencher`), and `paste` is a compile-time-only procedural macro. The `rand` advisory is more nuanced — it only affects code paths that obtain randomness through the global `rand::rng()` / `thread_rng()` entry point while a custom logger is installed. Security-critical randomness (key generation, signing nonces, the DKG Round 1 seed) in the audited crates is sourced from explicit RNGs and seeds, not from the global thread RNG, so the advisory is not exploitable in the audited code paths. The note is recorded here for completeness and to flag the dependency that should be monitored for upstream fixes.

Recommendation: Track the upstream of `dudect-bencher` (or replace it with a maintained alternative based on `clap 4.x`) to clear the `ansi_term` and `atty` advisories. Track `rand` for a release that addresses RUSTSEC-2026-0097, and continue to source security-critical randomness from explicit RNGs rather than `rand::rng()`. The `paste` advisory will be cleared once Plonky3 migrates off it; no local action is required.

Status: Reported for completeness. None of the advisories are exploitable in the audited code paths.

References

[Bie+25] Alexander Bienstock et al. *Quorus: Efficient, Scalable Threshold ML-DSA Signatures from MPC*. Cryptology ePrint Archive, Paper 2025/1163. 2025. URL: https://eprint.iacr.org/2025/1163.

[Bor+25] Giacomo Borin et al. *Threshold Signatures Reloaded: ML-DSA and Enhanced Raccoon with Identifiable Aborts*. Cryptology ePrint Archive, Report 2025/1166. 2025. URL: https://eprint.iacr.org/2025/1166.

[Cel+26] Sofía Celi et al. *Efficient Threshold ML-DSA*. Cryptology ePrint Archive, Paper 2026/013. 2026. URL: https://eprint.iacr.org/2026/013.

[Nat15] National Institute of Standards and Technology. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. Federal Information Processing Standards Publication 202. Aug. 2015. DOI: 10.6028/NIST.FIPS.202. URL: https://csrc.nist.gov/pubs/fips/202/final.

[Nat24] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. Federal Information Processing Standards Publication 204. Aug. 2024. DOI: 10.6028/NIST.FIPS.204. URL: https://csrc.nist.gov/pubs/fips/204/final.

Appendices

A Test Coverage

We measured the test coverage of the audited source code using `cargo llvm-cov`. Table 2 reports the line coverage per file, grouped by crate. Overall, the code base achieves **93.42%** line coverage and **85.86%** function coverage across the three audited crates, which we consider strong. Rows are colored according to line coverage: green for $\geq 90\%$, yellow for 70–90%, and red for $< 70\%$. As noted in Section 1.2, we still recommend extending the test suite to exercise additional corner cases, which is supported by the fact that the error files have a weak coverage.

Table 2: Per-file line coverage as reported by `cargo llvm-cov`.

File	Line coverage
`dilithium`	

Continued on next page

File	Line coverage
src/errors.rs	0.00%
src/fips202.rs	100.00%
src/lib.rs	83.67%
src/ml_dsa_87.rs	90.34%
src/ntt.rs	100.00%
src/packing.rs	99.24%
src/poly.rs	96.73%
src/polyvec.rs	98.24%
src/reduce.rs	100.00%
src/rounding.rs	100.00%
src/sign.rs	99.38%
hdwallet	
src/hderive.rs	87.23%
src/lib.rs	68.75%
src/test_vectors.rs	100.00%
src/tests.rs	78.74%
src/wormhole.rs	100.00%
threshold	
src/broadcast.rs	98.04%
src/circl_ntt.rs	100.00%
src/config.rs	89.25%
src/derivation.rs	100.00%
src/error.rs	64.49%
src/keygen/dealer.rs	96.65%
src/keygen/dkg/mod.rs	93.02%
src/keygen/dkg/protocol.rs	86.83%
src/keygen/dkg/state.rs	65.74%
src/keygen/dkg/types.rs	90.56%
src/keys.rs	79.35%
src/lib.rs	85.00%
src/participants.rs	99.66%
src/protocol/partial_pk.rs	100.00%
src/protocol/primitives.rs	97.40%
src/protocol/secret_sharing.rs	89.18%
src/protocol/signing.rs	92.86%
src/resharing/protocol.rs	85.55%
src/resharing/types.rs	91.58%
src/signer.rs	81.91%
src/signing_protocol.rs	92.01%
Total	**93.42%**

HashCloak